Main Concepts of the Project

-The project basically simulates how multiple users interact in a chatroom at the same time, using the concept of Thresads and Multithreading.(The core idea of the project)

-In the project each user that you see in the output is a seperate thread.

-All the threads run concurrently.

Multithreading:

-Multithreading is a technique where multiple threads run simultaeously within a single program.

-A thread is the smallest unit of execution

-Multiple threads use the same memory but run independently.

-EX:

Single thread → doing one task at a time
Multithreading → talking, typing and thinking all at the same time

-Why Multithreading is important:

-Concurrency: Multiple tasks done at the same time
-Better performance: CPU is utilized efficiently

-Thread lifecycle:

1.New → created
2.Runnable → ready to run
3.Running → executing
4.Terminated → finished

Synchronization:

-It is a mechanism to control access to shared resorces so that only one thread can use it at a time.

-Synchronization is used in the ChatRoom class methods to ensure that only one thread accesses shared resources at a time, preventing race conditions and ensuring correct chat output.

Code Explaination

1.ChatRoom.java:

-Import java.text.SimpleDateFormat;
This line imports the SimpleDateFormat class, which is used to format date and time into a readable string format like hours, minutes, and seconds.

-Import java.util.*;
This imports all utility classes such as Map, HashMap, Set, HashSet, and Date which are used throughout the program.

-Class ChatRoom {
This defines the ChatRoom class, which acts as the shared resource where all user threads interact.

-Private boolean running = true;
This variable controls whether the chat is active or not. If true, the chat continues; if false, all threads stop execution.

-Private SimpleDateFormat sdf = new SimpleDateFormat("HH:mm:ss");
This creates a formatter to display time in hours, minutes, and seconds format.

-Private int messageId = 1;
This variable is used to assign a unique ID to each event or message in the chat.

-Private int totalEvents = 0;
This keeps track of the total number of events that occur in the chat, including messages, typing, joining, and leaving.

-Private Map<String, Integer> userMsgCount = new HashMap<>();
This creates a HashMap to store the number of messages sent by each user. The key is the username, and the value is the message count.

-Private Set<String> users = new HashSet<>();
This creates a HashSet to store unique users in the chat. It ensures no duplicate users are stored.

-Private String startTime = "";
This variable stores the time when the chat starts.

-Private String endTime = "";
This variable stores the time when the chat ends.

-Public ChatRoom() {
This is the constructor of the ChatRoom class, which runs when a ChatRoom object is created.

-System.out.printf("%-10s %-4s %-10s %-10s %-10s %-10s %-25s\n",
This prints a formatted header row for the chat log table.
"TIME", "ID", "THREAD", "USER", "STATUS", "ACTION", "MESSAGE");
These are the column names printed in the table header.

-System.out.println("----------");
This prints a separator line below the header for better readability.

-Private String getTime() {
This method is used to get the current time.

-Return sdf.format(new Date());
This returns the current system time formatted using the SimpleDateFormat object.

-Private String getThread() {
This method returns the ID of the current thread.

-Return "T-" + Thread.currentThread().getId();
This fetches the current thread ID and formats it with a prefix "T-".

-Private void markStartTime() {
This method sets the start time of the chat.

-If (startTime.isEmpty()) {
This checks if the start time has not been set yet.

-StartTime = getTime();
If empty, it sets the start time to the current time.

-Public synchronized void userOnline(String user) {
This method is synchronized to ensure only one thread executes it at a time. It is used when a user joins the chat.

-MarkStartTime();
This ensures the start time is recorded when the first user joins.

-Users.add(user);
This adds the user to the set of active users.

-PrintRow(user, "ONLINE", "JOIN", "-");
This logs the event that the user has joined the chat.

-Public synchronized void userOffline(String user) {
This synchronized method logs when a user leaves the chat.

-PrintRow(user, "OFFLINE", "LEAVE", "-");
This prints the leave event for the user.

-Public synchronized void typing(String user) {
This synchronized method logs when a user is typing.

-PrintRow(user, "TYPING", "...", "-");
This prints the typing status in the log.

-Public synchronized void sendMessage(String user, String msg) {
This synchronized method handles sending messages from users.

-PrintRow(user, "ACTIVE", "SEND", msg);
This logs the actual message sent by the user.

-UserMsgCount.put(user, userMsgCount.getOrDefault(user, 0) + 1);
This updates the message count for the user. If the user is not present, it starts from 0.

-Private void printRow(String user, String status, String action, String msg) {
This method is used to print a formatted row for every event in the chat.

-System.out.printf("%-10s %-4d %-10s %-10s %-10s %-10s %-25s\n",
This prints the formatted output row.

-GetTime(),
This prints the current time.

-MessageId,
This prints the current message ID.

-GetThread(),
This prints the thread ID.

-User,
This prints the username.

-Status,
This prints the current status (ONLINE, OFFLINE, etc.).

-Action,
This prints the action performed (JOIN, SEND, etc.).
-Msg);
This prints the message content.

-MessageId++;
This increments the message ID for the next event.

-TotalEvents++;
This increments the total number of events.

-Public synchronized void stopChat() {
This synchronized method stops the chat safely.

-Running = false;
This sets the running flag to false, stopping all threads.
-
EndTime = getTime();
This records the time when the chat ended.

-System.out.println("\n===== CHAT ENDED =====\n");
This prints a message indicating the chat has ended.

-PrintSummary();
This calls the method to display the summary of the chat.

-Private void printSummary() {
This method prints the overall summary of the chat.

-System.out.println("================ SUMMARY ====================\n");
This prints the summary header.

-Int totalMsgs = 0;
This initializes a variable to count total messages.

-For (int count : userMsgCount.values()) {
This loops through all message counts.

-TotalMsgs += count;
This adds each user's message count to the total.

-System.out.println("Total Messages : " + totalMsgs);
This prints the total number of messages.

-System.out.println("Total Events Logged : " + totalEvents);
This prints the total number of events.

-System.out.println("Total Users : " + users.size());
This prints the total number of users.

-System.out.println("\nMessages Per User:");
This prints a heading for user-wise message count.

-For (String user : users) {
This loops through all users.

-Int count = userMsgCount.getOrDefault(user, 0);
This gets the message count for each user.

-System.out.printf("%-7s : %d\n", user, count);
This prints each user’s message count.

-String topUser = "";
This stores the most active user.

-Int max = 0;
This stores the highest message count.

-For (String user : userMsgCount.keySet()) {
This loops through users in the map.

-Int count = userMsgCount.get(user);
This gets each user's message count.

-If (count > max) {
This checks if the current user has the highest messages.

-Max = count;
This updates the maximum count.

-TopUser = user;
This updates the most active user.

-System.out.println("\nMost Active User : " + topUser + " (" + max + ")");
This prints the most active user.

-System.out.println("Chat Started At : " + startTime);
This prints the start time.

-System.out.println("Chat Ended At : " + endTime);
This prints the end time.

-System.out.println("\n==");
This prints the closing line of the summary.

-Public boolean isRunning() {
This method checks whether the chat is still running.

-Return running;
This returns the current state of the chat.

2.User.java:

-Import java.util.Random;
This imports the Random class, which is used to generate random numbers for delays and message selection.

-Class User extends Thread {
This defines the User class as a thread, meaning each user runs independently.

-Private String userName;
This variable stores the name of the user.

-Private ChatRoom chatRoom;
This holds the reference to the shared ChatRoom object where all users interact.

-Private Random random = new Random();
This creates a Random object to generate random delays and select random messages.
-
Private String[] messages = {
This defines an array of predefined messages that users can send.
"Hey guys!",
A sample message.
"What's up?",
A sample message.
"Anyone working on the project?",
A sample message.
"This is actually fun ",
A sample message.
"Did you complete the assignment?",
A sample message.
"Let’s submit it today",
A sample message.
"I’m tired bro ",
A sample message.
"Same here!",
A sample message.
"Let’s take a break",
A sample message.
"Back to work!"
A sample message.
};
This closes the messages array.

-Public User(String userName, ChatRoom chatRoom) {
This is the constructor for the User class.

-This.userName = userName;
This assigns the provided username to the userName variable.

-This.chatRoom = chatRoom;
This assigns the shared ChatRoom reference.

-SetName("Thread-" + userName);
This sets the name of the thread for identification.

-@Override
This indicates that the run() method overrides the method from the Thread class.

-Public void run() {
This method contains the code that will execute when the thread starts.

-ChatRoom.userOnline(userName);
This notifies the chat room that the user has joined.

-While (chatRoom.isRunning()) {
This loop keeps running as long as the chat is active.

-Try {
This block is used to handle exceptions.

-Thread.sleep(2000 + random.nextInt(3000));
This pauses the thread for 2 to 5 seconds randomly to simulate user delay.

-If (random.nextInt(4) == 0) continue;
This gives a 25% chance to skip sending a message, simulating idle behavior.

-ChatRoom.typing(userName);
This indicates that the user is typing.

-Thread.sleep(1500 + random.nextInt(2500));
This pauses the thread to simulate typing delay.

-} catch (InterruptedException e) {
This catches any interruption during sleep.

-E.printStackTrace();
This prints the error details.

-If (!chatRoom.isRunning()) break;
This checks if the chat has stopped and exits the loop if true.

-String msg = messages[random.nextInt(messages.length)];
This selects a random message from the messages array.

-ChatRoom.sendMessage(userName, msg);
This sends the selected message to the chat room.

-Try {
Another try block for delay after sending a message.

-Thread.sleep(1000 + random.nextInt(2000));
This pauses the thread for 1 to 3 seconds.

-} catch (InterruptedException e) {
Handles interruption.

-E.printStackTrace();
Prints error details.
}
End of while loop.

-ChatRoom.userOffline(userName);
This notifies the chat room that the user has left.
}
End of run method.
}
End of User class.

3.ChatSimulation.java:

-Import java.util.Scanner;
This imports the Scanner class to take input from the user.

-Public class ChatSimulation {
This defines the main class of the program.

-Public static void main(String[] args) {
This is the entry point of the Java program.

-System.out.println("\nPress ENTER anytime to stop the chat...\n");
This prints a message instructing the user how to stop the chat.

-ChatRoom chatRoom = new ChatRoom();
This creates a new ChatRoom object which will be shared among all users.

-User u1 = new User("Rahul", chatRoom);
This creates the first user thread with name Rahul.

-User u2 = new User("Priya", chatRoom);
This creates the second user thread.

-User u3 = new User("Arjun", chatRoom);
This creates the third user thread.

-U1.start();
This starts the first thread and calls its run() method.

-U2.start();
This starts the second thread.

-U3.start();
This starts the third thread.

-Scanner sc = new Scanner(System.in);
This creates a Scanner object to read input from the console.

-Sc.nextLine();
This waits for the user to press ENTER.

-ChatRoom.stopChat();
This stops the chat by setting the running flag to false.

-Sc.close();
This closes the Scanner to free resources.
}
End of main method.
}
End of ChatSimulation class.